专利申请技术交底书

（通用）

发明名称	一种面向权益证明共识算法的质押隐藏确定性抽签方法
申请人	北京理工大学
申报类型	■国家发明　□实用新型　□外观
发明人	张川、刘梦轩、祝烈煌、张超、高振鹏
技术联系人	刘梦轩、18600149604、mengxuanliu@outlook.com
特殊情况说明	

一、发明名称

【发明名称尽量清楚、简要，尽可能使用通用技术术语，不超过 25 个字】

一种面向权益证明共识算法的质押隐藏确定性抽签方法

二、技术领域

【请写出本申请提案中技术方案直接所属的技术领域。】

本发明属于区块链共识算法技术领域。

三、与本发明相关的背景技术（背景技术可以结合附图描述）

【本部分介绍如下内容：1、介绍现有技术的概况；2、介绍现有方案如何实施；其中，现有方案可以找一个或多个与本发明最接近的现有方案，具体描述；可引述现有专利文献、文章、论文、教科书等。】

区块链技术已经成为去中心化系统和应用的核心基础，其共识机制在保证系统安全性和一致性方面起着至关重要的作用。权益证明（Proof-of-Stake， PoS）作为一种广泛采用的共识机制，通过质押

代币来抽签选择领导者节点。然而，权益证明系统中公开抽签选择领导者节点的过程可能导致严重的安全问题。在领导者节点选出并打包出块期间，公开的领导者节点身份使其容易成为拒绝服务（DoS）攻击等恶意行为的目标，从而影响系统的正常运行。

为了应对这一挑战，一些研究[1-4]提出了秘密领导者选举（Secret Leader Election）方案，其中领导者节点的身份在出块之前保持秘密。例如，Algoran[3]和Ouroboros[4]等方案通过概率性选举机制来实现秘密领导者节点抽签，这些方案在一定程度上解决了权益证明系统中的领导者节点被攻击问题。然而，概率性抽签选举机制也带来了新的问题。由于中签结果的不确定性，可能出现0领导者节点产出，导致重选，或者多领导者节点产出，导致区块链分叉。这些问题会严重影响系统的稳定性和效率。对此，Boneh等人[5]于2020年提出了一种称为Single Secret Leader Election (SSLE)的确定性秘密领导者节点抽签选举方案。SSLE 确保每轮抽签选举产生唯一且确定的领导者节点，从而避免了概率性抽签选举中的0产出和分叉问题。然而，当前的确定性秘密领导者节点抽签选举方案[6-12]在应用于权益证明共识系统时都依赖于公开的质押分布。公开质押分布使得大质押节点暴露，且这些节点由于其高质押量有更大概率成为领导者节点，从而成为攻击的重点目标。目前虽然有一些隐藏质押分布的秘密领导者节点抽签选举方案[2][4]，通过使用零知识证明来证明用户生成的可验证随机函数（VRF）值与其质押数额的关系来实现隐私保护。然而这些方案仅在概率性抽签选举中有效，不适用于确定性秘密领导者节点抽签选举，因为确定性秘密领导者节点抽签选举并不依赖于VRF。因此迫切需要设计一种隐藏质押分布的确定性秘密领导者节点抽签选举方法，以解决现有方案中的不足，使其既能够实现对质押分布的隐私保护，又能够实现确定性的秘密领导者节点抽签选举，推动权益证明共识的发展。

四、现有技术的缺点及本发明所要解决的技术问题

【分析第三部分中提及的现有技术存在什么样的缺陷，提出本发明所要解决的技术问题。这个技术问题就是一篇专利的纲，下面全部的材料都是为解决这个技术问题而服务的。本发明解决不了的技术问题不写】

现有秘密领导者节点抽签选举方案，根据产出不同，分为概率性秘密领导者节点抽签选举[1-4]与确定性秘密领导者节点抽签选举[5-12]。

概率性秘密领导者节点抽签选举产出的领导者节点数量不固定，即可能出现零领导者节点产出或多领导者节点产出。出现零领导者节点的抽签选举结果就需要重选，而出现多领导者节点的抽签选举结果就需要广播通信来决定最终的领导者节点，在无边界的去中心化网络中，这一过程面临着分叉风险，分叉会为区块链带来巨大的损失。

确定性秘密领导者节点抽签选举产出的领导者节点数量固定，不会出现零产出或多产出的情况，从而杜绝了重选与分叉的风险。然而当前的确定性方案[5-12]在应用于权益证明共识的抽签选举时都需要依赖公开的权益分布表，而权益分布的泄露会让大权益节点成为攻击的重点目标。而当前能够隐藏质押分布的秘密领导者节点抽签选举方法[2][4]都是概率性的，无法适用于确定性秘密领导者节点抽签选举。

因此，本发明旨在解决现有方法的不足，提出一种能够隐藏质押分布的确定性秘密领导者节点抽签选举方法，在对质押分布进行隐私保护的同时，避免概率性抽签选举面临的重选与分叉的风险。该方法结合承诺技术、密钥同态的伪随机函数技术，秘密分享技术、零知识证明技术和门限全同态加密技术，在隐藏质押分布的同时，确保领导者节点抽签选举的确定性，不可预测性以及公平性。具体而言，该方法通过构建 Pedersen 承诺隐藏质押分布；通过全同态加密计算保障选举过程的机密性与确定性；通过结合秘密分享技术与密钥同态的伪随机函数技术构建分布式生成的伪随机数，以确保抽签选举的不可预测性；并基于 cut and choose 策略，结合秘密分享技术构建零知识证明方案，保证加密选举的密文输入与质押承诺的一致性，以及分布

式伪随机数的正确性，以保障选举的公平性。

五、本申请技术方案的详细阐述

本部分应该提供两块内容——具体实施方案、附图。篇幅不限，尽量详尽地描述，到本领域技术人员不需要付出创造性的劳动即可实施的程度。

其中：

（1）具体实施方案：

本部分应该详细描述本发明的实现过程。各类型专利必须写明的内容：

- 公式类、模型类案件写明公式推导原理，或建模原理，说明改进后公式、模型的由来；
- 机械类写明组成、连接关系、位置关系、工作原理；（必须有图）
- 电路类写明组成、连接关系，工作原理；（必须有图）
- 系统类写明组成、每个模块的功能，模块间的交互关系，工作原理（必须有图）；
- 方法类依据流程图写明方案，步骤间需要有联系，每个步骤均写明实现过程，如果是现有技术步骤可以简单描述。
- 文字没有办法解释清楚的问题，举例、配图。

注意，同一个技术术语在整个交底书里只用一个词来表达，不要用多个词来表达。对于英文缩写，括号中提供中文和英文全称。

（2）附图要求：

1、只能采用黑白线条图，可以是二维图或三维图，不能有阴影，不要用照片；

2、图的作用是辅助理解，不能只有图，图的内容应该在文字中描述；

3、附图中出现的各种符号，应该在具体实施方案的描述中有明确说明；

4、图的线条应该清晰，以缩小到 2/3 仍能看清为准。

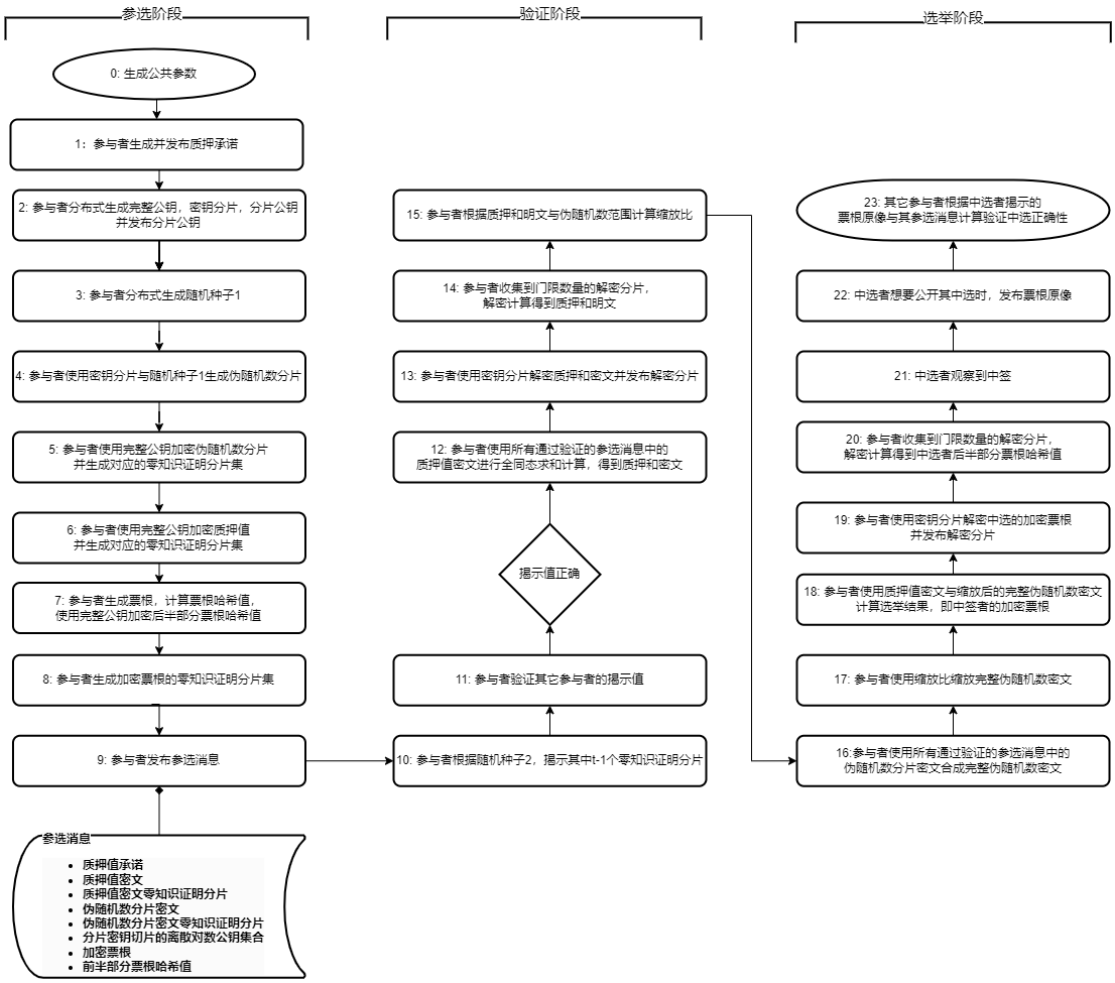

图 1 为本发明方法实施示意图

具体实施方式

下面结合附图对本发明做进一步详细说明。

具体地，一种面向权益证明共识算法的质押隐藏确定性抽签方法，流程详细分为 22 个步骤：

假设抽签选举有 T 个参与者，分别为$P_1, \ldots, P_T$。

步骤 0：生成公共参数，定义函数；

0. 生成公共参数 Setup(1^λ) -> pp (λ, p, q, G, g, h, N, σ, μ, ν)；其中λ为设置的安全参数，并根据λ生成公共参数。公共参数包括:

P: 一个λ位的大素数;

q: $p-1$ 的一个大素数因子。

G: 一个模为 p，阶为 q 的循环子群。用于 Pedersen 承诺

G': 另一个模为 p, 阶为 q 的循环子群。用于离散对数加密

g: 循环群 G 中的一个生成元.

h: G 中的另一个生成元，满足$\boldsymbol{h} = \boldsymbol{g}^{\boldsymbol{x}}$。 其中 x 是在$Z_q^*$中随机选择的一个秘密值。

g': 循环群 G'中的一个生成元.

k: 基于 TRLWE（基于 Torus 环的学习误差）问题的 FHE 算法中私钥向量长度;

m: 基于 LWE 的密钥同态 PRF 中随机矩阵的行数, m>>k;

N: 基于 TRLWE 问题的 FHE 算法中使用的二进制多项式环$\boldsymbol{B}[\boldsymbol{x}]/(\boldsymbol{x}^{\boldsymbol{N}}+\mathbf{1})$与 Torus 多项式环$\boldsymbol{T}_{\boldsymbol{q}}[\boldsymbol{x}]/(\boldsymbol{x}^{\boldsymbol{N}}+\mathbf{1})$的阶数。

σ: 基于 TRLWE 问题的 FHE 算法中，用于噪声采样的高斯分布标准差。为确保噪声不会影响解密时最终的近似取值，应选择合适的 σ，确保噪声的计算累加小于离散$\boldsymbol{T}_{\boldsymbol{q}}$上量化步长的一半（Quantization Step）。

μ: 基于 TRLWE 问题的 FHE 算法中，用于噪声采样的高斯分布均值。

ν(μ,σ2): 基于 TRLWE 问题的 FHE 算法中，用于噪声采样的高斯分布。

1. 函数定义:

ThFHE: 基于 TRLWE 问题的门限全同态加密方案。由 6 个概率多项式时间（PPT）算法组成：

$$ThFHE = (ThFHE.Setup, ThFHE.Enc, ThFHE.Eval, ThFHE.PartDec, ThFHE.FinDec)$$

设有 N'个参与方为：Pt={Pt1, …, PtN'}，门限为 n',则有：

$ThFHE.Setup(\lambda, N', n') \rightarrow (PK, sk1, \ldots skN')$:输入中$\lambda$为安全参数，输出为一个公钥 PK，以及 N'个部分私钥；

$ThFHE.Enc(PK, x) \rightarrow Cx$: 输入中 x 为秘密值，PK 为公钥，输出为全同态加密后的密文值；具体加密过程见[14]

$ThFHE.Eval(PK, Circuit, Inputs) \rightarrow Cy$:输入中 PK 为公钥，Circuit 为计算电路，Inputs 为计算电路的输入，输出 Cy 为计算结果；

其中，计算电路包括：

Ccompare($C_x, C_y = (C_{y1}, \ldots, C_{yN'}) \rightarrow (C_{x<y1}, \ldots, C_{x<yN'})$ ：输入中C_x为一个密文，C_y为一个密文向量，包含 N'个密文；输出为一个密文向量，向量中的第 i 个元素为 x<yi 的比较结果密文，比较结果的明文为 1 或 0，x<yi 则为 1 的密文，x>=yi 则为 0；

Clocate $(C_{r1}, \ldots, C_{rN'}) \rightarrow (CL_1, CL_2 \ldots, CL_{T'})$ ：输入中 $C_{r1}, \ldots, C_{rN'}$为 N'个密文，输出为长度为 N'的密文向量 $(CL_1, CL_2 \ldots, CL_{T'})$，向量中第一个元素$CL_1$=$C_{r1}$，除第一个元素外，第 i 个元素$CL_i$=$C_{ri} - C_{ri-1}$；

Cselect($(C_1, C_2 \ldots, C_{N'})$,$(C_{x1}, \ldots, C_{xN'})$) → Cti: 输入为两个 N' 长度的密文向量，输出 Cti 为两个密文向量的点积；

$ThFHE.PartDec(PK, Cts, SKi) \rightarrow Dts_{piecei}$: 输入为公钥 PK，密文 Cts,以及部分密钥 SKi，输出 Dts 的解密分片Dts_{piecei}；

$ThFHE.\mathrm{FinDec}(\mathrm{PK}, \mathrm{Cts}, Dts_{pieces}) \rightarrow \mathrm{ts}$: 输入为公钥 PK，密文 Cts，以及 n'个部分解密分片Dts_{pieces},输出为解密结果明文 ts;

SecretSharing: 线性门限秘密分享函数。设有一个秘密值 sv，有 N'个参与方，门限为 n'，则:

$$SecretSharing(sv) = (sv_{piece1}, \ldots, sv_{pieceN'})$$

具体地，如使用 shamir 秘密分享方案时:

1） 在 Zq 上选择一个 n'-1 阶多项式:

$$f(x) = \sum_{i=1}^{n'} a_{i-1} x^{i-1}$$

其中，a_0=sv;

2) 在 f(x)上选择 N'个点，如(1, f(1)), (2,f(2)) ,…, (N', f(N'));这 N'个点即为 sv 的 N'个秘密分片sv_{pieces};

凑齐任意 n'个 sv 的秘密分片($sv_{piece-r1}, \ldots$, $sv_{piece-rn'}$)，其中，(r1, ⋯,rn') ∈ （1, ⋯,N'）则有秘密还原函数:

$$SecretRecover(sv_{piece-r1}, \ldots, sv_{piece-rn'}) = \mathrm{sv}$$

具体地，如使用 shamir 秘密分享方案时:

$$sv = f(0) = \sum_{i=1}^{n'} f(i) \prod_{j=1 j\neq i}^{n'} \frac{0-x_j}{x_i-x_j}$$

当在指数上还原时:

$$sv = \prod_{i=1}^{n'} f(i)^{\prod_{j=1 j\neq i}^{n'} \frac{0-x_j}{x_i-x_j}}$$

在参选阶段：

步骤 1：每个参与者 Pi 生成并发布质押承诺 CM_i。

具体地，参与者在Z_q中随机选择一个随机数 R_i，并根据自己的质押值 STK_i 生成质押值的 Pedersen 承诺：$CM_i = g^{STK_i} h^{R_i}$；参与者公开发布CM_i。

步骤 2：参与者们分布式生成完整公钥，解密密钥分片，计算分片公钥并发布分片公钥；

具体地，参与者们分布式生成全同态加密密钥 DKG(pp, t, T)->(PK, SK_1, …, SK_T): pp 为 Setup 中生成的公共参数， t 为解密门限。具体操作如下：

1. $\forall Pi \in (P_1, P_2, \ldots, P_T)$： 在$B[x]/(x^N+1)$上随机选取 k 个 N-1 阶多项式生成 FHE 的部分私钥：

$$PSKi = (PSKi_1(x) = \sum_{j=1}^{N} PSKi_{1,j} \times x^{j-1}, \ldots, PSKi_k(x) = \sum_{j=1}^{N} PSKi_{k,j} \times x^{j-1})$$

2. Pi ： 在$T[x]/(x^N+1)$上随机选取 k 个 N-1 阶多项式得到用于生成公钥的 k 维向量， 并公开（广播）：

$$Ai = (Ai_1(x) = \sum_{j=1}^{N} Ai_{1,j} \times x^{j-1}, \ldots, Ai_k(x) = \sum_{j=1}^{N} Ai_{k,j} \times x^{j-1})$$

3. Pi：计算：$A_{sum} = \sum_{i=1}^{T} Ai$

$$= (Asum_1 = \sum_{i=1}^{T} Ai_1(x), \ldots, Asum_k = \sum_{i=1}^{T} Ai_k(x))$$

4. Pi：在$T[x]/(x^N+1)$上取 k 个极小的噪声多项式，其中，多项式的系数

在高斯分布ν(μ,σ²)中采样，生成 k 维噪声向量：

$$ei = (ei_1(x) = \sum_{j=1}^{N} ei_{1,j} \times x^{j-1}, \ldots, ei_k(x) = \sum_{j=1}^{N} ei_{k,j} \times x^{j-1})$$

5. Pi：生成部分私钥 PSKi 对应的 FHE 公钥 PPKi 并公开：

$$Bi_j(x) = Asum_j \bullet PSKi_j(x) + ei_j(x)\ j \in (1,2,\ldots,k)$$

$$PPKi = ((Asum_1(x), Bi_1(x)), \ldots, (Asum_k(x), Bi_k(x)))$$

6. Pi：计算完整公钥：

$$PK = \left(\left(Asum_1(x), Bsum_1 = \sum_{i=1}^{T} Bi_1(x)\right), \ldots, \left(Asum_k(x), Bsum_k = \sum_{i=1}^{T} Bi_k(x)\right)\right)$$
$$= ((PK1_A, PK1_B), \ldots, (PKk_A, PKk_B))$$

7. Pi：使用秘密分享算法生成 PSKi 的 T 个秘密分片：

$$SecretSharing(PSKi) = (PSKi_{share1}, \ldots, PSKi_{shareT})$$

PSKi 的第 j 个秘密分片：

$$PSKi_{sharej} = (PSKi_{1-sharej}, \ldots, PSKi_{k-sharej})\ j \in (1,2,..,T)$$

凑齐任意 t 个 $PSKi_{share}$ 即可以还原出 PSKi。

8. Pi：从生成的 T 个 PSKi 的秘密分片中，按序号选取 T-1 个 $PSKi_{share}$ 分发给对应的其余解密参与者（如 Pj）。并计算发布 T 个秘密分片的离散对数公钥：

$$(PPKi_{share1} = g'^{PSKi_{share1}}, \ldots, PPKi_{share1} = g'^{PSKi_{shareT}})$$

9. 所有解密参与者分发秘密分片完成后，Pi 持有的解密密钥分片为：

$$SKi = (PSK1_{sharei}, \ldots, PSKT_{sharei})$$
$$= ((PSK1_{1-sharei}, \ldots, PSK1_{k-sharei}), \ldots, (PSKT_{1-sharei}, \ldots, PSKT_{k-sharei}))$$
$$= \begin{pmatrix} PSK1_{1-sharei} & \cdots & PSKT_{1-sharei} \\ \vdots & \ddots & \vdots \\ PSK1_{k-sharei} & \cdots & PSKT_{k-sharei} \end{pmatrix}$$

对应的 FHE 密钥分片 k 维向量为：

$$SKi = (\sum_{j=1}^{T} PSKj_{1-sharei}, \ldots, \sum_{j=1}^{T} PSKj_{k-sharei})$$

10. Pi：计算并发布解密密钥分片的分片公钥：

$$PPK_i' = \left(\prod_{j=1}^{T} g'^{PSKj_{1-sharei}}, \dots, \prod_{j=1}^{T} g'^{PSKj_{k-sharei}}\right)$$

11 Pi: 根据其它参与者验证其它参与者发布的分片公钥；

步骤 3：分布式生成随机种子sd_1；

具体地，每个参与者 Pi 在Z_q上选取一个随机值Rnd_i;

$$sd_1 = Rnd_1\ xor\ Rnd_2\ xor \dots xor\ Rnd_T$$

步骤 4：参与者 Pi 根据随机种子sd_1生成伪随机数分片 PPRFi;

具体地，参与者使用密钥同态的伪随机函数 Flwe[13]生成$PPRFi = Flwe(SKi, sd_1)$;

1. 随机生成两个T_q上的 m×n 矩阵：A_0，A_1；
2. 将sd_1转化为二进制：$sd_1 = x_1x_2x_3 \dots x_u$，$u = \lceil log2(q) \rceil$为位数；
3. $Flwe(SKi, xd_1) = \lfloor \prod_{j=1}^{u} Axj \cdot SKi \rfloor_q$；其中当$x_j = 0$时，$Axj = A_0$；当$x_j = 1$时，$Axj = A_1$；

步骤 5：参与者使用完整公钥加密伪随机数分片并生成对应的 T’个零知识证明分片；

具体地，

1. 参与者使用完整公钥 PK，加密$PPRFi$，得到密文:

$$CPPRFi, = ThFHE.Enc(PK, PPRFi, r_{prf_i}, e_{prf_i})$$

2. 参与者 Pi 使用秘密分享算法生成SKi，r_{prf-i}, e_{prf-i}的 T’个分片：

$$SKi_{pieces} = SecretSharing(SKi) = (SKi_{piece1}, \ldots, SKi_{pieceT'})$$

其中，SKi_{piecel}=($SKi, 1_{piecel}$, … , SKi, k_{piecel})=($SecretShare - l(\sum_{j=1}^{T} PSKj_{1-sharei})$, ⋯, $SecretShare - l(\sum_{j=1}^{T} PSKj_{k-sharei})$);

$$r_{prf_{i-pieces}} = SecretSharing\left(r_{prf_i}\right) = (r_{prf_{i-piece1}}, \ldots, r_{prf_{i-pieceT'}})$$

$$e_{prf_{i-pieces}} = SecretSharing\left(e_{prf_i}\right) = (e_{prf_{i-piece1}}, \ldots, e_{prf_{i-pieceT'}})$$

凑齐任意 t'个SKi_{piece}，$r_{prf_{i-pieces}}$，$e_{prf_{i-pieces}}$ 即可以还原出SKi，r_{prf_i}，e_{prf_i}。

3. 参与者 Pi 生成SKi_{pieces}对应的离散对数公钥：

$$PPK_i{}'_{pieces} = (PPK_i{}'_{piece1}, \ldots, PPK_i{}'_{pieceT'})$$

其中，$PPK_{i_{piecel}} = (g'^{SKi,1_{piecel}}, \ldots, g'^{SKi,k_{piecel}})$;

4. 参与者生成伪随机数分片$CPPRFi$的 T'个分片：

$$CPPRF_{i_{Pieces}} = (CPPRF_{i_{piece1}}, \ldots, CPPRF_{i_{pieceT'}})$$

其中，第 L 个分片：

$$CPPRF_{i_{pieceL}} = ThFHE.Enc(PK, \prod_{j=1}^{u} Axj \cdot SKi_{pieceL}, r_{prf_{i-pieceL}}, e_{prf_{i-pieceL}})$$

凑齐任意 t'个 $CPPRF_{i_{piecej}}$，即可以还原出 $CPPRFi = \prod_{j=1}^{u} Axj \cdot SecretRecover\left(CPPRF_{i_{piece-recv1}}, \ldots, CPPRF_{i_{piece-recvt'}}\right)_q = \lfloor \prod_{j=1}^{u} Axj \cdot SKi \rfloor_q$。

步骤 6：参与者使用完整公钥加密质押值并生成对应的 T'个零知识证明分片；

具体地，

1. 参与者 Pi 使用完整公钥 PK 加密质押值。生成质押密文：

$$CSTKi = ThFHE.Enc(PK, STKi, r_{enc_i}, e_{stki})$$

其中，$STKi$为 Pi 的质押值，r_{enc_i}为随机数，e_{stki}为加密噪

声。

2. 参与者 Pi 使用秘密分享算法切割$STKi$，r_{enc_i}和e_{stki}，生成 T'个分片：

$$STKi_{pieces} = SecretSharing(STKi) = (STKi_{piece1}, \dots, STKi_{pieceT'})$$

$$r_{enc_{i-pieces}} = SecretSharing(r_{enc_i}) = (r_{enc_{i-piece1}}, \dots, r_{enc_{i-pieceT'}})$$

$$e_{stki_{pieces}} = SecretSharing(e_{stki}) = (e_{stki_{piece1}}, \dots, e_{stki_{pieceT'}})$$

3. 参与者 Pi 使用$STKi_{pieces}$、$r_{enc_{i-pieces}}$与$e_{stki_{pieces}}$生成$CSTKi$的 T'个分片：

$$CSTKi_{Pieces} = (CSTKi_{piece1}, \dots, CSTKi_{pieceT'})$$

凑齐任意 t'个$CSTKi_{piece}$，即可以还原出$CSTKi$。

其中，

$$CSTKi_{piecej} = ThFHE.Enc(PK, STKi_{piece1}, r_{enc_{i-piecej}}, e_{stki_{piecej}})$$

4. 参与者 Pi 使用秘密分享算法生成质押承诺CMi的 T'个分片：

   1. 参与者 Pi 使用秘密分享算法生成质押承诺随机数R_i的 T'个分片：

   $$R_{i_{pieces}} = SecretSharing(R_i) = (R_{i_{piece1}}, \dots, R_{i_{pieceT'}})$$

   2. 参与者 Pi 根据$STKi$分片与R_i分片生成质押承诺CMi的 T'个分片：

   $$CM_{i_{Pieces}} = (CM_{i_{piece1}}, \dots, CM_{i_{pieceT'}})$$

   其中，$CM_{i_{piecej}} = g^{STK_{i_{piecej}}} h^{R_{i_{piecej}}}$；

步骤 7：参与者生成票根，计算票根哈希值，并使用完整公钥加密后半部分票根哈希值；

具体地，

1. 参与者 Pi 在Z_q中随机选择一个随机数为票根，记为Ticket_i。
2. 参与者 Pi 计算票根哈希值：$\mathrm{Hticket}_i = \mathrm{Hash}(\mathrm{Ticket}_i)$。
3. 参与者 Pi 将票根哈希值$\mathrm{Hticket}_i$从中间位置分开：

$$\mathrm{Hticket}_i -> (\mathrm{Hticket}_{i_L}, \mathrm{Hticket}_{i_R})$$

4. 参与者 Pi 使用完整公钥加密后半部分票根哈希值：

$$Cticket_i = TRLWE.Enc(PK, \mathrm{Hticket}_{i_R}, r_{ticket_i}, e_{ticket_i})$$

其中，r_{ticket_i}加密所使用的随机数，e_{ticket_i}为加密所添加的噪声；

步骤 8：参选者生成加密票根的零知识证明分片集；

具体地：

1. 参与者 Pi 使用秘密分享算法切割$\mathrm{Hticket}_{i_R}, r_{ticket_i}, e_{ticket_i}$,分别生成 T'个分片；

$$\mathrm{Hticket}_{i_{R-pieces}} = SecretSharing(\mathrm{Hticket}_{i_R})$$
$$= (\mathrm{Hticket}_{i_{R-piece1}}, \dots, \mathrm{Hticket}_{i_{R-pieceT'}})$$

$$r_{ticket_{i-pieces}} = SecretSharing(r_{ticket_i}) = (r_{ticket_{i-piece1}}, \dots, r_{ticket_{i-pieceT'}})$$

$$e_{ticket_{i-pieces}} = SecretSharing(e_{ticket_i}) = (e_{ticket_{i-piece1}}, \dots, e_{ticket_{i-pieceT'}})$$

2. 参与者 Pi 根据$\mathrm{Hticket}_{i_R}, r_{ticket_i}, e_{ticket_i}$生成$Cticket_i$的 T'个分片；

$$Cticket_{i-pieces} = (Cticket_{i-piece1}, \dots, Cticket_{i-pieceT'})$$

凑齐任意 t'个$Cticket_{i-piece}$，即可以还原出$Cticket$。

其中，

$Cticket_{i-piecej} = ThFHE.Enc(PK, \mathrm{Hticket}_{i_{R-piece1}}, r_{ticket_{i-piece1}}, e_{ticket_{i-piece1}})$

步骤 8：参选者发布参选消息；

具体地，

参选者 Pi 的参选消息$\mathrm{RegisMsg}_i$ ={

质押承诺：CM_i，

质押值密文：$CSTKi$，

质押值密文零知识证明分片集：

质押值密文分片集：$(CSTKi_{piece1},\ldots,CSTKi_{pieceT'})$

质押承诺分片集：$(CM_{i_{piece1}},\ldots,CM_{i_{pieceT'}})$

伪随机数分片密文：$CPPRFi$

伪随机数分片密文零知识证明集：

$(CPPRFi_{piece1},\ldots,CPPRFi_{pieceT'})$，

分片密钥切片的离散对数公钥集合：

$(PPK_i{}'_{piece1},\ldots,PPK_i{}'_{pieceT'})$，

加密票根（加密的后半部分票根哈希值）：$Cticket_i$，

加密票根零知识证明分片集：

$(Cticket_{i-piece1},\ldots,Cticket_{i-pieceT'})$，

前半部分票根哈希值：$\mathrm{Hticket}_{i_L}$

}

在验证阶段：

步骤 9：分布式生成随机种子sd_2；

生成方式与步骤 3 相同。

步骤 10：参与者根据随机种子sd_2，揭示其中 t'-1 个零知识证明分片。

具体地，参与者 Pi 根据随机种子sd_2，使用伪随机数生成器(如梅森旋转算法)生成[1, T']内 t'-1 个不同的伪随机数。即 PRNG(sd_2, T', t'-1) -> （Idx_1…, Idx_{t-1}）；参与者 Pi 发布对应的 t'-1 个分片的揭示值：如Idx_1对应的分片为：质押值密文零知识证明分片$CSTKi_{Idx_1}$与质押承诺零知识证明分片$CM_{i_{piece-Idx_1}}$，伪随机数分片密文零知识证明分片$CPPRFi_{piece-Idx_1}$，加密票根零知识证明分片$Cticket_{i-piece-Idx_1}$；

步骤 11：参与者 Pi 验证其它参与者(例如 Pj)的揭示值;

具体地，在验证密文质押值的真实性时：

在验证 Pj 的伪随机数分片密文$CPPRFj$的真实性时：

0. 如序号为Idx_1的$CPPRFj$切片$CPPRFj_{piece-Idx_1}$的揭示值为：

$$CPPRFj_{piece-Idx_1-reveal}$$
$$= (SKj_{piece-Idx_1}, r_{prf_{j-piece-Idx_1}}, e_{prf_{j-piece-Idx_1}})$$

1. 验证伪随机数分片密文零知识证明是否能还原$CPPRFj$：

$$\text{SecretRecover}(CPPRFj_{piece1},\ldots,CPPRFj_{pieceT'}) \text{ ?= } CPPRFj$$

2. 验证分片密钥切片的离散对数公钥集合是否对应 Pj 在步骤 2-10 中发布的离散对数分片公钥：

$$\text{SecretRecover}\left(PPK_j{}'_{piece1},\ldots,\ PPK_j{}'_{pieceT'}\right) ?= \ PPK_j{}'$$

注：此处的 SecretRecover 时指数上的还原方案;

3. 使用通过验证的分片密钥切片的离散对数公钥集合$\left(PPK_j{}'_{piece1},\ldots,\ PPK_j{}'_{pieceT'}\right)$，验证揭示的 t'-1 个伪随机数分片

密文零知识证明分片中，生成伪随机数分片的密钥分片是否与其离散对数公钥一致：

（如序号为Idx_1的切片时）

$$g'^{SKj_{piece-Idx_1}} ?= PPK_j'_{piece-Idx_1};$$

4. 验证揭示的 t'-1 个伪随机数分片密文零知识证明分片中，加密是否正确：

$$ThFHE.Enc\left(PK, SKj_{piece-Idx_1}, r_{prf_{j-piece-Idx_1}}, e_{prf_{j-piece-Idx_1}}\right) ?= CPPRFj_{piece-Idx_1}$$

其中，，$e_{prf_{j-piece-Idx_1}}$应在噪声范围内。

在验证加密票根的真实性时：

0. 如序号为Idx_1的$Cticket_j$切片$Cticket_{j_{piece-idx1}}$的揭示值为：

$$Cticket_{j_{piece-Idx_1-reveal}} = (\mathrm{Hticket}_{j_{R-piece-Idx_1}}, r_{ticket_{j-piece-Idx_1}}, e_{ticket_{j-piece-Idx_1}})$$

1. 验证伪随机数分片密文零知识证明是否能还原$Cticket_j$：

$$\mathrm{SecretRecover}(Cticket_{j-piece1}, \dots, Cticket_{j-pieceT'}) ?= Cticket_j$$

2. 验证揭示的 t'-1 个加密票根密文零知识证明分片中，每个分片的加密是否正确：

如验证序号为Idx_1的切片时：

$$ThFHE.Enc\left(PK, \mathrm{Hticket}_{j_{R-piece-Idx_1}}, r_{ticket_{j-piece-Idx_1}}, e_{ticket_{j-piece-Idx_1}}\right) ?= Cticket_{j_{piece-Idx_1}}$$

其中，$e_{ticket_{j-piece-Idx_1}}$应在噪声范围内；

在验证质押值密文的正确性时：

0. 如序号为Idx_1的质押密文$CSTKj$切片$CSTKj_{piece-idx1}$的揭示值为：

$$CSTKj_{piece-Idx_1-reveal} = (STKj_{piece-Idx_1}, r_{enc_{j-piece-Idx_1}}, e_{stkj_{piece-Idx_1}});$$

序号为Idx_1的质押承诺CM_j切片$CM_{j_{piece-idx1}}$的揭示值为：

$$CM_{j_{piece-Idx_1-reveal}} = (STK_{j_{piece-Idx_1}}, R_{j_{piece-Idx_1}});$$

其中，序号相同的质押密文揭示值中的$STKj_{piece}$应与质押承诺揭示值中的$STKj_{piece}$一致；

1. 验证质押密文零知识证明分片集是否能够还原$CSTKj$：

$$SecretRecover(CSTKi_{piece1}, \ldots, CSTKi_{pieceT'})? = CSTKj$$

2. 验证质押承诺零知识证明分片集是否能够还原CM_j：

$$SecretRecover(CM_{i_{piece1}}, \ldots, CM_{i_{pieceT'}})? = CMj$$

其中，此处的 SecretRecover 是在指数上的还原方案；

3. 验证揭示的 t'-1 个质押密文零知识证明分片中，每个分片的加密是否正确：

如验证序号为Idx_1的切片时：

$$ThFHE.Enc\left(PK, STKj_{piece-Idx_1} r_{enc_{j-piece-Idx_1}}, e_{stkj_{piece-Idx_1}}\right) ? = CSTKj_{piece-idx1}$$

其中，$e_{stkj_{piece-Idx_1}}$应在噪声范围内；

4. 验证揭示的 t'-1 个质押密文零知识证明分片中，每个分片的计算是否正确：

如验证序号为Idx_1的切片时：

$$g^{STK_{j_{piece-Idx_1}}} h^{R_{j_{piece-Idx_1}}} ? = CM_{j_{piece-Idx_1-reveal}}$$

步骤 12：参与者使用所有通过验证的参选消息中的质押值密文进行全同态求和计算。

具体地，若全部 T'个参选消息都通过验证，则：

$$Csum = \sum_{i=1}^{T'} CSTKi$$

步骤 13：参与者使用密钥分片解密质押和密文并发布解密分片。

具体地，参与者 Pi 计算并发布：

$$Dsum_{piecei} = ThFHE.PartDec(PK, Csum, SKi)$$

步骤 14：参与者收集到门限数量的解密分片，解密计算得到质押和的明文。

具体地，设参与者 Pi 收集到的 t'个解密分片为$(Dsum_{piece-i1},\ldots,Dsum_{piece-it})$，则：

$$Sum = ThFHE.FinDec(PK, Csum, (Dsum_{piece-i1}, \ldots, Dsum_{piece-it}))$$

步骤 15：参与者根据质押和的明文与伪随机数范围计算缩放比例。

具体地，缩放比计算如下：

$$scale = \frac{Sum}{q-1}$$

步骤 16：参与者使用所有通过验证的消息中的伪随机数分片密文同态合成完整伪随机数密文。

具体地，若 T'个参选消息全部通过验证，则完整伪随机数密文为：

$$CPRF = SecretRecover(CPPRF1, \dots, CPPRFT')$$

步骤 17：参与者使用缩放比，缩放完整伪随机数密文。

具体地：

$$CPRF_{scaled} = scale * CPRF$$

步骤 18：参与者使用质押值密文与完整伪随机数密文计算选举结果，即中签者的加密票根。

具体地，选举计算过程如下：

1) 按照质押的顺序（即序号），计算质押密文累加值：

$$(C_1, C_2, \dots, C_{T'}) = (CSTK1, CSTK1 + CSTK2, \dots, \sum_{i=1}^{T'} CSTKi)$$

2) 使用缩放后的完整伪随机数密文，与累加值密文进行同态比较：

$$(C_{<1}, C_{<2}, \dots, C_{<3})$$
$$= ThFHE.Eval(PK, Ccompare, (CPRF_{scaled}, (C_1, C_2, \dots, C_{T'})))$$

3) 确定中签者位置：

$$(CL_1, CL_2, \dots, CL_{T'})$$
$$= ThFHE.Eval(PK, Clocate, (C_{<1}, C_{<2}, \dots, C_{<3}))$$

4) 计算中签信息:

$$C_{leaderTicket}$$
$$= ThFHE.Eval(PK, Cselect, ((Cticket_{i-piece1}, \dots, Cticket_{i-pieceT'}), (CL_1, CL_2, \dots, CL_{T'})))$$

步骤 19：参与者使用密钥分片解密中选的票根密文并发布解密分片。

具体地，参与者 Pi 计算并发布：

$$DLeaderTicket_{piecei} = ThFHE.PartDec(PK, C_{leaderTicket}, SKi)$$

步骤 20：参与者收集到门限数量的解密分片，解密计算得到中选者后半部分票根哈希值。

具体地，设参与者 Pi 收集到的 t' 个解密分片为 $(DLeaderTicket_{piece-i1}, \ldots, DLeaderTicket_{piece-it'})$，则：

$$\mathrm{Hticket}_{leader_R} = ThFHE.FinDec(PK, C_{leaderTicket}, (DLeaderTicket_{piece-i1}, \ldots, DLeaderTicket_{piece-it'}))$$

步骤 21：中选者观察到自己中签。

具体地，假设 Pi 为中签者，Pi 观察到：

$$\mathrm{Hticket}_{i_R} = \mathrm{Hticket}_{leader_R}$$

步骤 22：中选者想要公开其中选时，发布票根原像。

具体地，假设 Pi 为中签者，揭示时，Pi 公开Ticket_i。

步骤 23：其它参与者根据中选者揭示的票根原像与其参选消息计算验证中选的正确性。

具体地，假设 Pi 为中签者，验证 Pi 中选揭示的正确性：

$$Hash(\mathrm{Ticket}_i) ?= \mathrm{Hticket}_{i_L} + \mathrm{Hticket}_{leader_R}$$

六、本申请的关键点和欲保护点

【本部分是基于第五部分提炼出技术方案的关键创新点，列出 1、2、3……以提醒代理人注意，便于专利代理人撰写权利要求书。简单点明即可】

1、本发明提出了一种面向权益证明共识算法的能够隐藏质押分布的单一秘密领导者节点抽签选举方案。

2、本发明构建了一种基于 Cut & Choose 策略的零知识证明方案，能够证明 Pedersen 承诺与全同态密文输入的一致性。

3、本发明结合密钥同态的伪随机数生成函数与秘密分享技术，构建了一种分布式伪随机数生成方案，避免了在全同态密文计算中生成伪随机函数所需的乘法深度。

七、与第三条中最接近的现有技术相比，本申请有何技术优点

【这部分结合发明点，详细分析本发明为什么能够解决前面提到的技术问题。如果本申请取得了更多的技术效果也请列出。注意：要结合方案分析，切忌空谈效果，让代理人明白技术——效果之间的关系。可以分（1）（2）（3）条撰写。】

1、本发明能够在隐藏质押分布的情况下，进行确定性秘密领导者节点抽签选举。相比于概率性秘密领导者节点抽签选举方案，本发明的方案避免了重选与分叉风险。

2、本发明相比于其它确定性抽签方案，避免了因公开质押分布造成的隐私泄露问题，避免了大质押节点成为重点攻击目标。

3、本发明相比于其它基于门限全同态加密的确定性选举方案，通过设计使用一种分布式伪随机数全同态密文生成方法，减少了在全同态密文下生成伪随机数的乘法深度，减少了计算开销。

八、针对第五部分中的技术方案，是否还有别的替代方案？（可选项）

【如果有，请详尽写明，内容的提供可以扩大专利的保护范围，防止他人绕过本技术去实现同样的发明创造；可以是部分结构、器件、方法步骤的替代，也可以是完整的技术方案的替代，例如：两个部件的连接为卡式连接，但铰链连接也可以，因此铰链连接即为替代方案。】

九、其他有助于理解本申请提案的技术资料（可选项）

【可以提供辅助代理人对技术方案更好更快理解的技术资料，比如相关的术语解释、协议、标准、论文。】

相关文献：

[1] Sarah Azouvi, Patrick McCorry, and Sarah Meiklejohn. Betting on blockchain consensus with fantomette. CoRR, abs/1805.06786, 2018.

[2] Chaya Ganesh, Claudio Orlandi, and Daniel Tschudi. Proof-of-stake protocols for privacy-aware blockchains. IACR Cryptology ePrint Archive, 2018.

[3] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In SOSP, 2017.

[4] Thomas Kerber, Markulf Kohlweiss, Aggelos Kiayias, and Vassilis Zikas. Ouroboros crypsinous: Privacy preserving proof-of-stake. IACR Cryptology ePrint Archive, 2018.

[5] Boneh, D., Eskandarian, S., Hanzlik, L., & Greco, N. (2020, October). Single secret leader election. In *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies* (pp. 12-24).

[6] Catalano, D., Fiore, D., & Giunta, E. (2022, July). Adaptively secure single secret leader election from ddh. In Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (pp. 430-439).

[7] Backes, M., Berrang, P., Hanzlik, L., & Pryvalov, I. (2022, September). A framework for constructing single secret leader election from MPC. In European Symposium on Research in Computer Security (pp. 672-691). Cham: Springer Nature Switzerland.

[8] Freitas, L., Tonkikh, A., Bendoukha, A. A., Tucci-Piergiovanni, S., Sirdey, R., Stan, O., & Kuznetsov, P. (2023). Homomorphic sortition–single secret leader election for pos blockchains. Cryptology ePrint Archive.

[9] Burdges, J., Alper, H. K., Stewart, A., & Vasilyev, S. (2023). Sassafras and semi-anonymous single leader election. *Cryptology ePrint Archive*.

[10] Boneh, D., Partap, A., & Rotem, L. (2023). Post-Quantum Single Secret Leader Election (SSLE) from Publicly Re-Randomizable Commitments. Cryptology ePrint Archive.

[11] Catalano, D., Fiore, D., & Giunta, E. (2023, May). Efficient and universally composable single secret leader election from pairings. In IACR International Conference on Public-Key Cryptography (pp. 471-499). Cham: Springer Nature Switzerland.

[12] Zhai M, Wu Q, Liu Y, et al. Secret Multiple Leaders & Committee Election with Application to Sharding Blockchain[J]. IEEE Transactions on Information Forensics and Security, 2024.

[13] Boneh D, Lewi K, Montgomery H, et al. Key homomorphic PRFs and their applications[C]//Annual Cryptology Conference. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013: 410-428.

[14] Chillotti I, Gama N, Georgieva M, et al. TFHE: fast fully homomorphic encryption over the torus[J]. Journal of Cryptology, 2020, 33(1): 34-91.